



Architecting Android Multimedia

⇒ Chapter 7: Audio, Video, and Camera Use

A developer's guide to playback, recording, hardware manipulation, and resource management.

The Multimedia Framework Blueprint

1. Playback (MediaPlayer, VideoView)

2. Low-Latency (SoundPool)

3. Hardware Access (AudioTrack, AudioRecord)

4. Capture (Camera, MediaRecorder)



The Enemy: ANR (Application Not Responding)

- ⇒ Displayed by the **Android platform** when an app performs too many functions on the main thread.
- ⇒ Heavy media processing blocks the thread, triggering an ANR.
- ⇒ **Proper use of asynchronous media state machines** prevents this failure.
- ⇒ **Proper use of asynchronous media state machines** prevents this failure.

Standard Playback Solutions



⇒ MediaPlayer: The core class for audio/video playback from URIs, local files, or raw application resources.

```
=> MediaPlayer.create(this,  
R.raw.sample_audio);
```

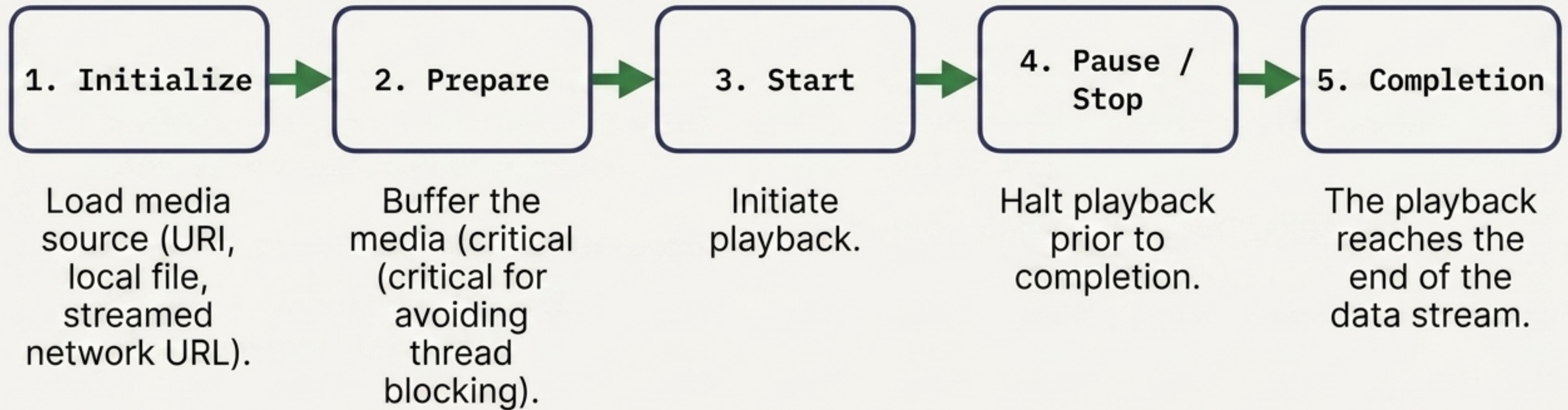
```
=> mediaPlayer.start();
```



⇒ VideoView: Handles video files and provides display options like scaling and tinting.

⇒ MediaController: Attaches to VideoView to provide automatic UI controls (seek, pause, forward).

The MediaPlayer State Machine



Supported Native Media Formats

=> Codec support dictates playback capabilities on specific Android hardware.

Audio Formats

- => **AAC:** Standard for mobile streaming (Supported via 3GP, MP4 containers).
- => **MP3:** Mono/Stereo standard, CBR or VBR.
- => **FLAC:** Lossless audio (Android 3.1+).
- => **MIDI:** Ringtone and low-overhead formats (.mid, .ota).
- => **PCM/WAVE:** Raw, uncompressed audio (up to 44100 Hz).

Video Formats

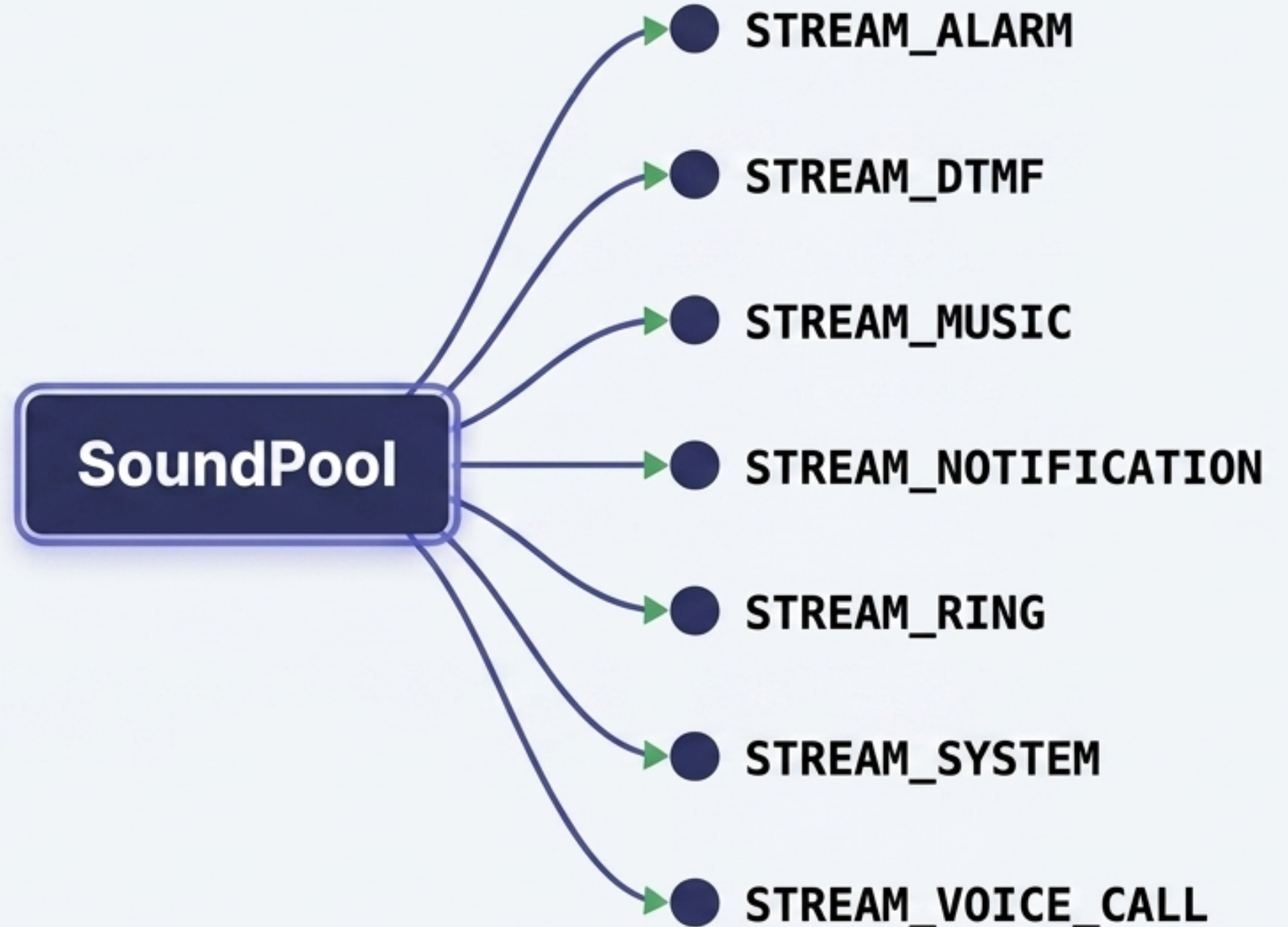
- => **Core Codecs:** H.263, H.264, MPEG-4, V8.
- => **Containers:** 3GP, MKV, MP4, TS, WEBM.

Low-Latency Audio via SoundPool

=> Problem: MediaPlayer is too slow for games or UI sound effects.

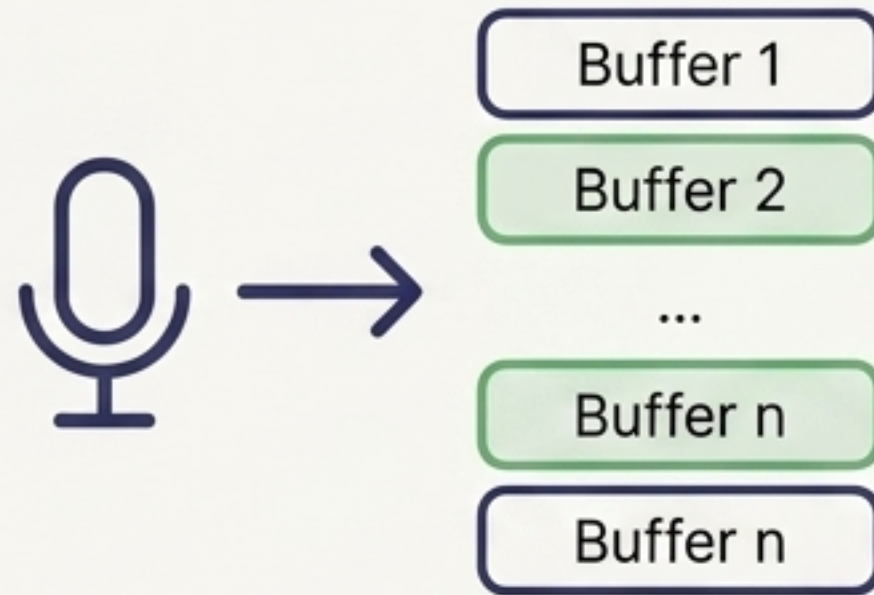
=> Solution: SoundPool preloads sample collections into memory from APKs or folders.

=> Optimizes resource management for multiple simultaneous audio streams.



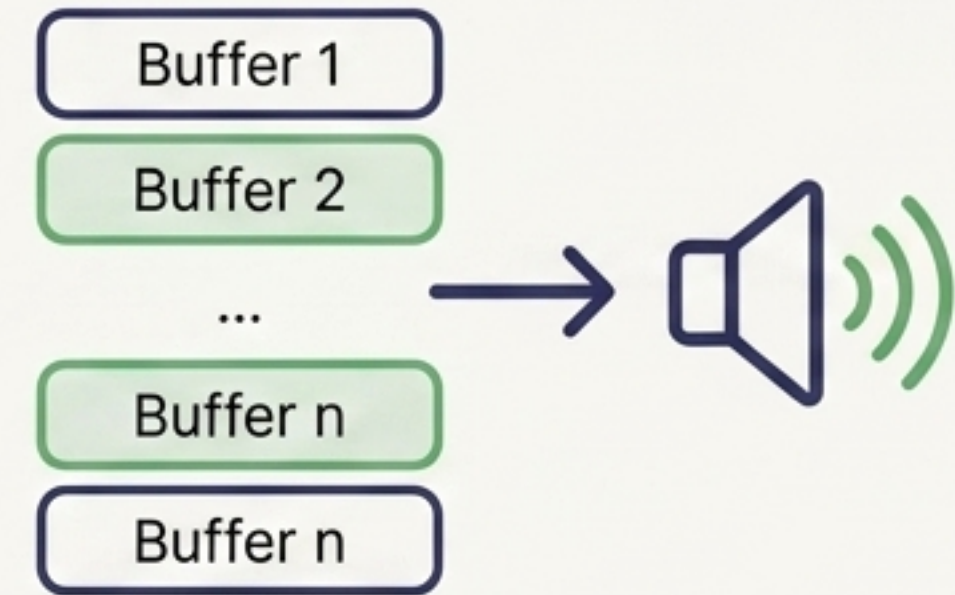
Direct Hardware Audio Buffers

- => Standard encoding formats (MP3/AAC) can introduce unwanted processing overhead.
- => Android provides two classes for raw, uncompressed manipulation.



1. AudioRecord

=> Records raw audio **directly** from the device's hardware buffers.



2. AudioTrack

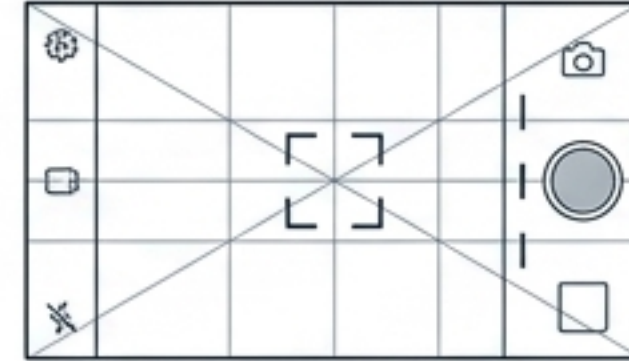
=> **Directly plays** raw audio data (PCM) **directly into** the hardware output buffers.

Camera Integration: Two Approaches



Approach: Camera Intent

- ⇒ **Class:** `MediaStore.ACTION_IMAGE_CAPTURE`
- ⇒ **Benefit:** Easiest implementation. Delegates the work to the device's native camera app.
- ⇒ **Result:** Returns the finished image to `onActivityResult`.



Approach: Camera API

- ⇒ **Requirement:** Manifest permission `<uses-permission android:name="android.permission.CAMERA"/>`
- ⇒ **Benefit:** Direct hardware control to build fully custom camera UIs and adjust low-level settings.
- ⇒ **Critical Lifecycle:** Must call `Camera.open()` to initialize, and `camera.release()` when finished to free hardware resources.

Recording Video Capabilities



Approach: Video Intent

- ⇒ **Class:**
`MediaStore.ACTION_VIDEO_CAPTURE`
- ⇒ **Action:** Launches the native video recorder, allowing users to start, stop, review, and retake the video before returning to your app.



Approach: MediaRecording API

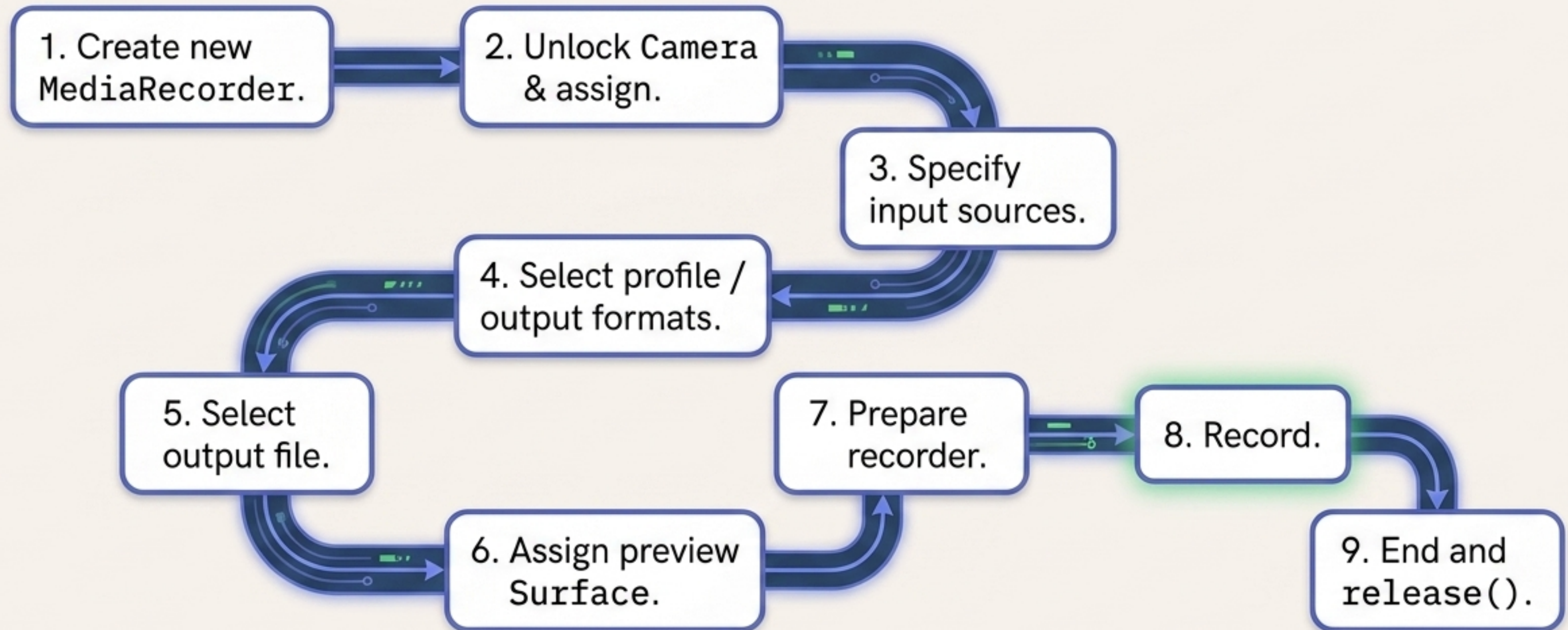
- ⇒ **Action:** Uses the `MediaRecorder` class for deep, custom integration.

Required Permissions:

1. `android.permission.CAMERA`
2. `android.permission.RECORD_AUDIO`
3. `android.permission.RECORD_VIDEO`

The MediaRecorder State Machine

=> MediaRecorder manages recording as a strict state machine. The order of configuration is mandatory.



Chapter 7 Synthesis: The Right Tool for the Job

The Developer Goal	The Optimal Solution	The API/Class
=> Play a podcast or stream a movie.	Standard Playback	MediaPlayer & VideoView
=> Build a fast-paced 2D game with laser sounds.	Pre-loaded Low Latency	SoundPool
=> Manipulate uncompressed audio waves.	Raw Hardware Access	AudioTrack & AudioRecord
=> Capture a quick user avatar photo.	Delegate to Native App	MediaStore Intents
=> Build a custom Snapchat-style video UI.	Full Hardware Control	MediaRecorder & Camera API